

Zero-TTT

将 MCTS 搜索经验压缩进双私有神经快记忆

个人研究项目

2026 年 8 月

摘要

本文给出 Zero-TTT 的核心研究设计。项目研究一个特定问题：能否在每次蒙特卡洛树搜索（MCTS）结束后，将搜索形成的策略与价值信息压缩进一份跨搜索保留的神经快记忆，使后续局面以更少搜索获得更好的判断。慢权重负责通用棋力；独立快记忆分支负责本局内由搜索产生的临时知识。黑白双方从同一份可学习初始化模板复制出彼此隔离的快记忆，并只使用自己的搜索结果写入自己的记忆。一次 MCTS 内所有参数冻结；搜索结束后只进行一次快权重更新，真实动作仍由本次更新前的搜索结果决定，随后搜索树被丢弃。

可学习初始化通过局部元学习获得：写入节点构成内层任务，未写入的搜索节点及该玩家下一次行棋的根局面构成外层任务。外层梯度精确穿过当前一次快权重更新，但不跨整盘传播高阶计算图。本文只规定这一方法的数学语义、信息边界和可证伪假设；所有可调工程规格与执行安排由可频繁更新的实施文档承担。

	慢策略-价值模型与独立非线性快记忆分支
核心对象	
	一份可学习模板，逐盘复制为黑白两份私有状态
记忆初始化	
	单次 MCTS 内权重冻结，搜索后集中写入一次
搜索协议	
	压缩本方 MCTS 的策略改进与价值修正
运行时目标	
	写入后泛化到未写入节点和下一次本方根局面
元训练目标	
	当前一次更新精确；跨真实回合截断高阶计算图
梯度边界	
	快记忆能否降低后续搜索所需的计算预算
主要判据	

目录

1 研究问题与主张边界	3
1.1 核心问题	3
1.2 不主张等价于完整搜索树	3
1.3 核心设计原则	3
2 慢模型与独立快记忆	4
2.1 状态、视角与慢模型	4
2.2 快记忆作为残差分支	4
2.3 什么是慢参数，什么是快状态	5
3 共同模板与黑白私有状态	5
3.1 从同一模板产生两个学习者	5
3.2 搜索拥有者决定整棵树使用哪份记忆	5
3.3 信息所有权	6
4 冻结搜索与搜索后单次写入	6
4.1 冻结搜索	6
4.2 搜索摘要	6
4.3 任务对齐的内层目标	7
5 可学习初始化与局部元训练	7
5.1 为什么初始化也是学习对象	7
5.2 同树保留目标	8
5.3 下一次本方根目标	8
5.4 单步精确、跨手截断	8
5.5 外层究竟训练什么	9
6 训练阶段与数据边界	9
6.1 阶段一：传统策略-价值基线	9
6.2 阶段二：快记忆元训练	9
6.3 阶段三：双私有记忆自博弈	9
7 可证伪假设与实验比较	10
7.1 核心假设	10
7.2 必要对照	11
7.3 黑白分歧作为诊断而非输入	11
8 局限、风险与未来方向	11
8.1 已知局限	11
8.2 后续研究方向	12
9 文档边界	12
10 总结	12

1 研究问题与主张边界

1.1 核心问题

传统 AlphaZero 类系统在一个局面上运行 MCTS，得到比原始网络更强的局部策略，然后把搜索结果作为训练数据留到以后使用。当前搜索树通常只服务于当前或紧邻局面；一旦树被丢弃，其中的大部分计算也随之消失。Zero-TTT 希望在两种极端之间增加一个中间层：

- 慢权重在许多棋局之间学习稳定、通用的棋力；
- MCTS 在当前局面进行昂贵而精确的显式规划；
- 快记忆在一盘棋内部，将已经完成的搜索压缩成可泛化的临时参数状态。

因此，项目的主要问题不是“快权重能否背下当前根节点的答案”，而是：

给定本方先前若干次 MCTS 的搜索摘要，快记忆能否改善未被直接写入的相关局面，并在本方下一次行棋时减少重新发现相同知识所需的搜索计算？

这种结构可以视为局内、在线的 Expert Iteration：树搜索承担慢而强的局部规划，神经记忆承担快而有损的归纳；归纳后的记忆又用于引导未来搜索 [2, 1]。

1.2 不主张等价于完整搜索树

“把 MCTS 内置到快权重中”在本文中是功能性简称，而不是数据结构上的等价。搜索树显式保存节点连接、访问计数、边价值和探索过程；固定容量神经网络只能学习这些信息的有损摘要。项目不要求从快记忆还原整棵树，也不主张一次写入能够替代任意预算的 MCTS。

快记忆可能具有的优势是跨局面泛化：一棵树只能精确复用其实际覆盖的节点，而神经记忆可能把某个局部搜索发现迁移到结构相似但未被搜索过的局面。该优势必须通过保留节点、后续真实局面以及树复用基线共同验证，而不能由训练损失下降直接推出。

1.3 核心设计原则

本文固定以下长期原则：

1. **搜索与写入分离**。单次搜索期间参数不变，避免同一棵树混合来自不同模型版本的先验和叶值。
2. **任务对齐写入**。快记忆直接学习 MCTS 改进后的策略与价值，而不是重建棋盘或执行与围棋决策无关的代理任务。
3. **私有而对称**。黑白使用独立运行时记忆，但二者来自同一模板并遵循相同更新规则。
4. **基础路径保留**。快记忆提供残差修正；即使记忆尚未学好，慢模型仍保留完整输出路径。
5. **初始化必须可学习**。模板不只是零矩阵或随机种子，而要被外层目标训练成适合快速吸收搜索经验的状态。

6. 主张可证伪。必须比较树复用、记忆重置和搜索预算，区分真正的搜索压缩与额外参数带来的表面收益。

2 慢模型与独立快记忆

2.1 状态、视角与慢模型

设围棋公开状态为 s ，合法动作集合为 $\mathcal{A}(s)$ ，当前行棋方为 $\text{turn}(s)$ 。慢权重记为 θ 。慢模型产生共享特征、基础策略 logits 和基础价值 logit：

$$F_\theta(s) = (h_\theta(s), l_\theta^{\text{base}}(s), a_\theta^{\text{base}}(s)).$$

价值始终采用当前节点行棋方视角。换手时，MCTS 负责翻转价值符号；快记忆不改变这条约定。

慢权重的职责是保存跨棋局稳定的棋形知识、策略先验和价值估计。其基础训练可以沿用普通策略-价值学习：

$$\mathcal{L}_{\text{slow}} = \text{CE}(\pi^{\text{target}}, p_\theta^{\text{base}}(s)) + \lambda_z \rho(v_\theta^{\text{base}}(s) - z),$$

其中 ρ 是稳健的价值损失， z 可以是终局或其他经过定义的价值目标。数据来源、目标细节和训练配方不属于本文的稳定定义。

2.2 快记忆作为残差分支

令 $c \in \{B, W\}$ 表示当前搜索的拥有者，而不是当前模拟节点的行棋颜色。节点 u 相对搜索拥有者的角色为

$$r_c(u) = \begin{cases} \text{self}, & \text{turn}(s_u) = c, \\ \text{opponent}, & \text{turn}(s_u) \neq c. \end{cases}$$

慢参数 ξ 定义查询投影、读出投影和安全门控。查询为

$$q_u = Q_\xi(h_\theta(s_u), r_c(u)),$$

独立快记忆网络以运行时参数 ϕ^c 读取该查询：

$$m_u = M_{\phi^c}(q_u).$$

快记忆产生策略与价值的残差，而不替代基础路径：

$$\begin{aligned} \Delta l_u &= P_\xi^\pi(m_u, r_c(u)), \\ \Delta a_u &= P_\xi^v(m_u, r_c(u)), \\ l_u &= l_\theta^{\text{base}}(s_u) + g_{\pi, \xi}(s_u) \odot \Delta l_u, \\ v_u &= \tanh(a_\theta^{\text{base}}(s_u) + g_{v, \xi}(s_u) \Delta a_u). \end{aligned}$$

非法动作在 softmax 前被屏蔽，得到 p_u 。门控 $g_{\pi, \xi}$ 与 $g_{v, \xi}$ 限制记忆分支的影响，并允许系统从接近基础模型的安全状态开始训练。

本文只要求 M_ϕ 是容量足够的独立非线性神经记忆，不固定它的层数、宽度或具体激活函数。线性矩阵记忆实现简单，但深层记忆能够表示更复杂的 key-value 关系；Titans、ATLAS 与 LaCT 的结果为使用大容量非线性记忆提供了直接动机 [5, 6, 7]。

2.3 什么是慢参数，什么是快状态

对象	变化时间尺度	主要职责
慢模型 θ	棋局之间	通用策略、价值与棋盘表示
记忆控制参数 ξ	棋局之间	决定写什么、怎样查询、怎样读出与门控
模板 ϕ_{init}	棋局之间	提供一种适合被一次搜索梯度快速更新的初始组织
运行时状态 ϕ^B, ϕ^W	每次本方搜索后	保存当前棋局中本方搜索形成的私有临时知识

模板属于 checkpoint 中可训练的长期参数；从模板复制出的 ϕ^B, ϕ^W 才是运行时快状态。称模板“可学习”意味着外层训练会优化它，而不是把运行时状态跨棋局永久保存。

3 共同模板与黑白私有状态

3.1 从同一模板产生两个学习者

每盘开始时，从同一个可学习模板建立两份独立状态：

$$\phi_0^B = \text{clone}(\phi_{\text{init}}), \quad \phi_0^W = \text{clone}(\phi_{\text{init}}).$$

两者初值相同，但存储互不共享。随着棋局进行，黑白只接收各自 MCTS 的写入信号，于是从共同起点形成不同的局内认知。外层训练中，来自黑方和白方的元损失共同更新同一个 ϕ_{init} ，使模板学习与颜色无关的“如何从自己的搜索中学习”。

棋盘输入仍应表达贴目、轮次等真实不对称因素；共享模板并不假装黑白处境相同。它只要求记忆的学习规则在交换颜色和相对视角后保持一致。

3.2 搜索拥有者决定整棵树使用哪份记忆

当白方在真实状态 s_t 发起 MCTS 时，整棵搜索树都使用 ϕ_t^W 。即使某个模拟节点轮到黑方走，也不能切换到真实的 ϕ_t^B ；此时 $r_W(u) = \text{opponent}$ ，表示“白方搜索内部对对手节点的评估”。黑方搜索同理。

这条规则同时满足两项要求：

- 一次 MCTS 的所有网络评价来自同一个模型状态；
- 一方无法通过模拟节点读取另一方真正的私有搜索经历。

角色标记不等同于显式对手建模。MVP 中的 `opponent` 节点仍然假设对手进行高质量博弈，只帮助同一记忆区分“我的决策节点”和“对方的应对节点”。针对某个对手行为偏好的建模属于后续扩展。

3.3 信息所有权

双方不得读取对方的快记忆、搜索树或搜索摘要。可读性与写入权限如下表所示。

信息	黑方可读	白方可读	可用于写入	说明
公开棋盘与真实动作	是	是	间接	双方共同环境
黑方搜索树与摘要	是	否	仅黑方	黑方私有计算
白方搜索树与摘要	否	是	仅白方	白方私有计算
黑方运行时快记忆	是	否	仅黑方	不向白方模拟节点开放
白方运行时快记忆	否	是	仅白方	不向黑方模拟节点开放
共同初始化模板	是	是	不在局内直接改	每盘只复制一次

对手完成真实落子后，自己的快记忆不立即执行一次“观察更新”。公开动作会通过新棋盘自然进入下一次本方搜索。这样可以把第一版研究问题保持为“从自己的搜索学习”，避免混入对手模仿和最佳响应学习。

4 冻结搜索与搜索后单次写入

4.1 冻结搜索

当前真实状态为 s_t ，行棋方为 c ，其运行时快记忆为 ϕ_t^c 。一次搜索可以抽象为

$$\mathcal{T}_t = \text{MCTS}(s_t; \theta, \xi, \phi_t^c, \omega_t),$$

其中 ω_t 表示搜索随机性。在 $\mathcal{T}_t$ 构建期间， θ 、 ξ 和 ϕ_t^c 全部冻结。MCTS 只消费网络给出的策略先验和节点价值，选择、扩展、备份和访问计数均不求导。

搜索完成后，根访问分布决定本手候选策略。真实动作 a_t 在任何快权重写入之前从该搜索结果中确定：

$$a_t \sim \Pi_{\text{play}}\left(\pi_{\text{root},t}^{\text{search}}\right).$$

因此，搜索后的写入只影响未来搜索，不回头改变已经完成的当前决策。

4.2 搜索摘要

从搜索树导出纯数据摘要

$$\mathcal{D}_t = \{e_u : u \in \mathcal{U}_t\},$$

每个节点条目至少表达

$$e_u = (s_u \text{ 或根相对路径}, r_c(u), \pi_u, V_u, N_u).$$

其中

$$\pi_u(a) = \frac{N_u(a)}{\sum_b N_u(b)}, \quad V_u = \sum_a \pi_u(a) Q_u(a).$$

V_u 采用节点当前行棋方视角, N_u 表示节点搜索置信度。搜索摘要为停止梯度目标; 本文不要求保存 Python 对象或完整树结构。

候选节点被划分为互不重叠的写入集与保留集:

$$\mathcal{D}_t = \mathcal{W}_t \dot{\cup} \mathcal{H}_t.$$

$\mathcal{W}_t$ 用来更新快记忆; $\mathcal{H}_t$ 不参与本次更新, 只用于衡量写入后的泛化。划分应兼顾访问置信度、搜索深度和角色覆盖, 但具体数量和采样规则属于实验配置。

4.3 任务对齐的内层目标

对于条目 e_u , 记当前快记忆下的策略和价值为 $p_u(\phi)$ 与 $v_u(\phi)$ 。访问权重归一化为

$$\alpha_u = \frac{w(N_u)}{\sum_{j \in \mathcal{W}_t} w(N_j)},$$

其中 w 是单调置信度函数。写入损失定义为

$$\mathcal{L}_t^{\text{write}}(\phi) = \sum_{u \in \mathcal{W}_t} \alpha_u [\text{CE}(\pi_u, p_u(\phi)) + \lambda_v \rho(v_u(\phi) - V_u)] + \lambda_a \mathcal{R}(\phi, \phi_{\text{init}}).$$

正则项 $\mathcal{R}$ 只表达“运行时记忆不能无界漂移”的稳定要求; 可以由锚定、归一化或其他等价约束实例化。关键语义是: 快记忆学习网络原始判断与 MCTS 改进结论之间的差异, 而不是泛化的棋盘重建任务。这种最终任务对齐的写入思想与 TTT-E2E、In-Place TTT 的动机一致 [4, 8]。

搜索完成后仅执行一次朴素更新:

$$\phi_{t+1}^c = \phi_t^c - \eta_t \nabla_{\phi} \mathcal{L}_t^{\text{write}}(\phi)|_{\phi=\phi_t^c},$$

其中 $\eta_t > 0$ 可以是外层学习的写入尺度。另一方状态保持不变:

$$\phi_{t+1}^{\bar{c}} = \phi_t^{\bar{c}}.$$

随后提交早已选定的 a_t , 销毁 $\mathcal{T}_t$ 。不存在 warm/final 双搜索, 也不在搜索中途更新参数。

5 可学习初始化与局部元训练

5.1 为什么初始化也是学习对象

如果 ϕ_{init} 永远是固定随机状态, 外层系统只能学习如何围绕一个偶然的参数组织进行写入。可学习模板的目标是形成一种适合快速学习的内部布局: 一次搜索梯度应把信息写入容易被后续相似查询读取的位置, 同时尽量不破坏其他判断。

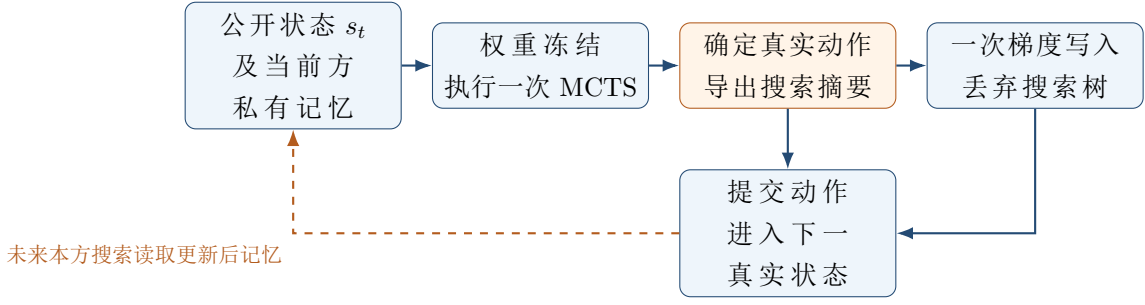


图 1: 真实回合协议。当前动作完全由冻结搜索决定；搜索后的快记忆写入只服务于未来。

原始 TTT 将可更新模型视为序列隐状态，并在外层学习 Key、Value、Query 等任务参数，使内部自监督任务真正服务于最终语言建模目标 [3]。Zero-TTT 采用相同的内外层区分，但用 MCTS 策略和价值替代 token 重建：

- 内层：当前一次搜索结束后，只改变运行时快记忆；
- 外层：跨许多搜索片段优化模板、查询/读出投影、门控和写入尺度。

5.2 同树保留目标

从当前状态写入得到适应后记忆 $\hat{\phi}_{t+1}^c$ 。未参与写入的 $\mathcal{H}_t$ 用来检验快记忆是否只背诵训练节点：

$$\mathcal{L}_t^{\text{hold}} = \sum_{u \in \mathcal{H}_t} \beta_u \left[\text{CE}(\pi_u, p_u(\hat{\phi}_{t+1}^c)) + \lambda_h \rho(v_u(\hat{\phi}_{t+1}^c) - V_u) \right].$$

如果写入损失下降而保留损失不改善，说明快记忆更接近局部拟合器，而不是搜索知识压缩器。

5.3 下一次本方根目标

设 $\tau(t, c) > t$ 是玩家 c 下一次真实行棋的时刻。两次本方行棋之间，对方真实动作改变公开棋盘，但不会写入 ϕ^c 。因此， $s_{\tau(t, c)}$ 是检验跨回合记忆最直接的局面。

使用下一次本方 MCTS 在根节点产生的停止梯度目标 $(\pi_\tau^{\text{root}}, V_\tau^{\text{root}})$ ，定义

$$\mathcal{L}_t^{\text{future}} = \text{CE}(\pi_\tau^{\text{root}}, p(s_\tau; \hat{\phi}_{t+1}^c)) + \lambda_f \rho(v(s_\tau; \hat{\phi}_{t+1}^c) - V_\tau^{\text{root}}).$$

这个目标问的是：上一手搜索写入后，记忆能否在对手真正落子造成局面变化后仍然提供帮助。它比在写入根节点上重复评分更接近项目的最终目的。

5.4 单步精确、跨手截断

运行时数值状态必须整盘保留，但把高阶计算图穿过整盘会使训练成本和数值风险迅速增加。为此，在回放第 t 手时使用重锚定状态

$$\bar{\phi}_t^c = \phi_{\text{init}} + \text{stopgrad}(\phi_t^c - \phi_{\text{init}}).$$

其前向值与真实运行时状态完全相同；反向时， $\partial \bar{\phi}_t^c / \partial \phi_{\text{init}} = I$ 。随后重新执行当前一次可微写入：

$$\hat{\phi}_{t+1}^c = \bar{\phi}_t^c - \eta_t \nabla_{\bar{\phi}_t^c} \mathcal{L}_t^{\text{write}}(\bar{\phi}_t^c).$$

外层目标为

$$\mathcal{L}_t^{\text{meta}} = \mathcal{L}_t^{\text{hold}} + \lambda_{\text{future}} \mathcal{L}_t^{\text{future}}.$$

梯度精确穿过当前一次内层梯度运算，因此包含当前写入损失的 Hessian 项：

$$\frac{\partial \hat{\phi}_{t+1}^c}{\partial \phi_t^c} = I - \eta_t \nabla_{\phi_t^c}^2 \mathcal{L}_t^{\text{write}}.$$

完成本手外层反向后释放该高阶图；下一手仍使用按真实更新累积的数值 ϕ_{t+1}^c 。因此，本方法称为**单步精确元梯度与跨手截断**：它精确回答“当前这一笔怎样写得更有用”，但忽略此前多次写入之间的高阶 Jacobian 链，不声称整盘精确元梯度。

5.5 外层究竟训练什么

在快记忆训练阶段，外层优化的主要对象是

$$\{\phi_{\text{init}}, \xi, \text{写入尺度及其他记忆控制参数}\}.$$

慢模型 θ 可以先冻结，使实验能够区分快记忆的真实贡献和基础棋力变化。只有当独立记忆闭环稳定且主要假设获得支持后，才考虑联合微调 θ 。复杂遗忘门、动量、Muon 或 Omega 类上下文更新器都不是核心定义；它们可以在相同内外层接口下作为后续替换 [5, 6, 7]。

6 训练阶段与数据边界

6.1 阶段一：传统策略-价值基线

先训练一个完全不依赖快记忆的标准策略与价值模型。该阶段旨在获得能够稳定指导 MCTS 的基础网络，并分别验证规则、策略合法性、价值视角和搜索正确性。快记忆分支不参与基础训练，或由关闭的输出门隔离。

训练数据只要求是许可清晰、语义可验证的高质量自博弈数据。具体来源、输入格式、辅助目标和数据规模留给实施阶段评估，不进入研究主文档。

6.2 阶段二：快记忆元训练

在可靠基线上运行冻结权重的 MCTS，收集搜索摘要。以 $\mathcal{W}_t$ 为内层写入任务，以 $\mathcal{H}_t$ 和下一次本方根为外层任务，训练 ϕ_{init} 与 ξ 。这一阶段优先冻结慢模型，从而回答更干净的问题：同一个基础网络能否因为局内记忆而更有效地利用过去搜索。

模板初值可以来自普通神经初始化，但残差输出门应允许从接近零修正的状态开始。随着元训练进行，模板本身会变成具有非零结构的“易学状态”，而不是始终被约束为零。

6.3 阶段三：双私有记忆自博弈

每盘从共同模板复制黑白记忆，双方轮流执行“冻结搜索-选择动作-本方写入-销毁树”。运行时状态整盘保留，终局后销毁。黑白产生的元训练片段应按搜索拥有者对称处理，避免先行方仅因多一次搜索而在外层目标中获得不成比例的权重。

联合更新慢模型、快记忆复杂优化器以及特定数据课程属于后续研究选择，而不是机制闭环成立的前提。

7 可证伪假设与实验比较

7.1 核心假设

H1: 同树泛化

写入 $\mathcal{W}_t$ 后， $\mathcal{H}_t$ 上的策略与价值误差应优于写入前；否则快记忆只是在背诵写入节点。

H2: 跨回合保留

在玩家下一次真实行棋根节点，保留记忆应优于重置为模板；否则记忆没有跨越对手真实动作产生实际价值。

H3: 搜索计算摊销

“较低搜索预算 + 已积累快记忆”应逐渐接近或超过“较高搜索预算 + 每手重置记忆”。这是“搜索被部分压缩”的主要操作性判据。

H4: 私有状态分化

黑白记忆应因不同私有搜索经历形成可测但有意义的差异；交换或重置其中一方时，其后续判断应发生系统性变化。

H5: 颜色对称

在交换颜色、棋盘视角与角色标记后，共同模板应表现出一致的学习能力；性能不应依赖两份不同的颜色模板。

7.2 必要对照

至少需要区分以下对照条件：

条件	搜索树	快记忆	回答的问题
标准基线	每手丢弃	无	没有跨手搜索信息时的基础表现
树复用基线	尽可能复用	无	显式缓存已经能保留多少搜索价值
重置记忆	每手丢弃	每手重置	额外分支本身是否有静态收益
私有持续记忆	每手丢弃	整盘保留	快记忆能否压缩并泛化过去搜索
高预算重置	每手丢弃	重置	记忆收益相当于多少额外搜索计算

如果私有持续记忆只优于“树丢弃”而不能接近“树复用”，结论应表述为补偿树信息损失，而不是优于显式搜索缓存。如果它在不属于旧子树的新局面上仍有优势，才更支持神经泛化解释。

7.3 黑白分歧作为诊断而非输入

研究阶段可以在同一公开局面上离线比较 ϕ^B 与 ϕ^W 的策略修正、价值修正和参数距离，用来分析双方是否形成不同的局内认知。正式行棋协议不能读取对方记忆来计算搜索预算或动作，否则会破坏私有边界。

未来可以训练一个只依赖本方历史的不确定度头，去预测离线观察到的高分歧局面；在此之前，黑白分歧只作为日志和解释工具。

8 局限、风险与未来方向

8.1 已知局限

- **搜索教师会自我强化：**MCTS 由同一个基础网络引导，其错误可能被快记忆快速放大。保留节点和高预算对照只能缓解，不能消除教师偏差。
- **大容量记忆可能漂移：**更大状态提高表达力，也增加梯度爆炸、遗忘旧搜索和压制基础路径的风险。安全门控与稳定约束是必要组成，但具体方案需实验决定。
- **元梯度是局部的：**重锚定忽略跨手高阶依赖，不能用于声称完整的整盘元优化。

- **双方共同适应**：黑白快状态都在变化，导致局内对手非平稳。严格私有写入减少直接追逐，但不能保证不存在策略振荡。
- **角色标记不是对手画像**：模拟对手节点表达的是本方搜索中的对手响应，不代表已经学习某个具体对手的行为偏好。
- **搜索摘要必然有损**：节点筛选与单步梯度会丢失树结构信息，是否保留了真正有用的抽象必须由实验回答。

8.2 后续研究方向

在核心机制成立后，可以沿以下方向扩展：

- 增加只从对方公开真实动作学习的小型对手观察区，并限制其只能修正 MCTS 的对手节点；相关方向可参考 BRExIt 的对手策略辅助建模 [10]。
- 进行记忆重置、黑白交换、局部区域交换和输出门关闭实验，识别快记忆实际保存的内容。
- 利用本方可观测的不确定度动态分配搜索预算，但不让正式玩家读取对方私有状态。
- 在策略和标量价值之外加入稀疏逐动作 Q 目标，研究它是否保留更多搜索结构。
- 比较 SGD、动量、归一化、Muon、数据依赖遗忘和 Omega 类局部上下文写入规则。
- 保存不同棋局中适应后的快记忆快照，形成临时策略种群并进行交叉对局。
- 研究以神经更新替代显式树的更激进方案；Policy Gradient Search 说明搜索期间适应神经策略具有可行性，但它不属于当前冻结搜索设计 [9]。

9 文档边界

本文是稳定的研究设计，不是实现规格。以下内容有意放在可频繁更新的 Markdown 文档中：

- 架构容量与所有数值超参数；
- 搜索、筛选、优化与性能方面的实验配置；
- 软件组织、运行环境、状态持久化和故障处理方案；
- 数据操作、阶段排期与工程验收流程。

只有当核心算法语义发生变化—例如改为搜索中更新、黑白共享运行时状态或取消可学习模板—才需要修改本文。普通超参数和实现选择应记录在实施计划、设计决策或开发日志中。

10 总结

Zero-TTT 将一盘棋中的学习划分为三个时间尺度：慢模型跨棋局学习通用棋力，MCTS 在当前局面进行显式规划，黑白私有快记忆在搜索之间保存各自规划的神经摘要。双方从同一份可学

习模板出发，却因私有搜索经历而逐渐分化；任何一方都不能读取另一方的搜索或记忆。

每次 MCTS 内权重固定，搜索后只写入一次，当前动作不受写入后的参数影响。模板通过写入集、同树保留集和下一次本方根局面组成的元任务进行训练；梯度精确穿过当前一次更新并跨手截断。项目最终是否成立，不取决于快记忆能否降低训练损失，而取决于它能否在未写入的未来局面上摊销过去 MCTS 的计算。

参考文献

- [1] David Silver, Julian Schrittwieser, Karen Simonyan, et al. Mastering the Game of Go without Human Knowledge. *Nature*, 550:354–359, 2017.
- [2] Thomas Anthony, Zheng Tian, David Barber. Thinking Fast and Slow with Deep Learning and Tree Search. *NeurIPS*, 2017.
- [3] Yu Sun, Xinhao Li, Karan Dalal, et al. Learning to (Learn at Test Time): RNNs with Expressive Hidden States. *ICLR*, 2025.
- [4] Arnub Tandon, Karan Dalal, Xinhao Li, et al. End-to-End Test-Time Training for Long Context. 2025.
- [5] Ali Behrouz, Peilin Zhong, Vahab Mirrokni. Titans: Learning to Memorize at Test Time. 2024.
- [6] Ali Behrouz, Zeman Li, Praneeth Kacham, et al. ATLAS: Learning to Optimally Memorize the Context at Test Time. 2025.
- [7] Tianyuan Zhang, Sai Bi, Yicong Hong, et al. Test-Time Training Done Right. 2025.
- [8] Guhao Feng, Shengjie Luo, Kai Hua, et al. In-Place Test-Time Training. 2025.
- [9] Thomas Anthony, Robert Nishihara, Philipp Moritz, Tim Salimans, John Schulman. Policy Gradient Search: Online Planning and Expert Iteration without Search Trees. 2019. <https://arxiv.org/abs/1904.03646>.
- [10] Daniel Hernandez, Hendrik Baier, Michael Kaisers. BRExIt: On Opponent Modelling in Expert Iteration. 2022. <https://arxiv.org/abs/2206.00113>.